

EDA: Energy-Efficient Inter-Layer Model Compilation for Edge DNN Inference Acceleration

Bo Ren Pao, I-Chia Chen[†], En-Hao Chang[‡], and Tsung Tai Yeh[†]

National Yang Ming Chiao Tung University[†], Google[‡]

{s311554020.cs11, apollo312581005.ii12, ttyeh14}@nycu.edu.tw, enhao@google.com

Abstract—Modern handheld devices often employ neural processing units (NPUs) to accelerate deep neural network (DNN) inference applications. Unlike the AI accelerator of a data center, the NPU of an edge device has strict price and energy budgets. In an NPU, DRAM memory consumes much energy due to frequent data movement across on-chip and off-chip memory. The inter-layer operator scheduling has been shown to reduce off-chip memory transactions by reusing DNN operator outputs on the on-chip memory space. However, it often deeply traverses operators and substantially increases off-chip memory data traffic when the on-chip memory of an NPU decreases in size. Consequently, this work creates a DNN model compilation framework called *EDA* that transparently improves the energy efficiency of an NPU by adjusting the operator traversal depth of the inter-layer operator scheduling in a stacked DNN model. First, *EDA* transforms a DNN model into a tensor-splitting model. Second, *EDA* breaks the tensor-splitting model graph into multiple subgraphs. Third, the *EDA* inter-layer cost model quickly determines the depth of each subgraph. Fourth, *EDA* properly manages the on-chip shared memory space of an NPU to avoid overusing memory space. Finally, *EDA* devises the operator grouping method to improve the MAC unit and on-chip memory space utilization. *EDA* improves the geometric means of $2.08\times$ and $2.39\times$, respectively, in energy efficiency and performance over the NPU with designated memory buffers.

I. INTRODUCTION

With the evolution of deep neural networks (DNNs) and domain-specific accelerators (DSA), modern handheld devices increasingly contain neural processing units (NPUs) to accelerate DNN inference applications [34], [41]. These modern NPUs usually contain on-chip memory, off-chip memory, a multiply-accumulate (MAC) unit, and an element-wise engine that tackles complicated operations such as activation functions. Unlike large-scale AI accelerators in a data center, the NPU within an edge device has strict constraints on the price and energy budgets. Previous work [39] has found that off-chip DRAM memory energy dominates the energy consumption of DNN inference applications in an NPU. In addition, the amount of off-chip memory traffic often determines the energy consumption of the off-chip memory. The DNN model compiler, which determines how DNN inference applications within an NPU are run, is used to decrease the amount of off-chip memory traffic by rescheduling the execution of the DNN model operators. As a result, the model compiler can reduce the energy consumption of the off-chip memory, and thus reduce the energy consumption of the entire NPU, conserving the battery life of a handheld device and ensuring the reliability of the system.

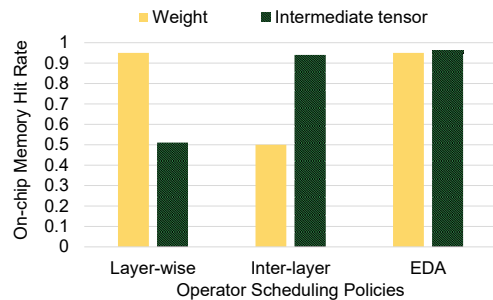


Fig. 1. On-chip Memory Hit Rate of Different Operator Schedulers

Challenges: The key to the ability of the DNN model compiler is to reduce off-chip memory traffic that lies in its inter-layer operator scheduler [9], [42], [62]. The model compiler first divides each tensor into several small tiles; these small tiles can fit in the memory of the chip. The inter-layer operator scheduler completes the operation in one tile before moving onto the operation of the next tile. Without the inter-layer operator scheduler, the NPU chip must continuously pull data from the off-chip memory due to the limited memory space on the chip itself. By reusing multiple small-tiled intermediate tensors (inputs and outputs) of the stacked DNN model in the on-chip memory, the inter-layer operator scheduler can avoid the long off-chip memory access latency. To maximize the data reuse of intermediate tensors in a DNN model, the inter-layer operator scheduler must traverse through the initial small tiles of all operators before moving on to the next set of small tiles. Therefore, all of the weights in a DNN model must still be saved in the on-chip memory. Because of the high number of operators, each traversal is often deep; as a result, the on-chip memory, due to its small size, cannot hold the entire DNN model weights. Hence, when the inter-layer operator traversal is deep, the model compiler will significantly increase the off-chip memory data traffic rather than reducing it.

The second challenge lies in the structure of on-chip memory buffers. Traditional NPUs [29], [30] save the operator inputs, outputs, and weights of a DNN model separately in designated on-chip memory buffers. However, because the tensors in a DNN model may be of varying lengths, the size of the tensors may not fit in their designated on-chip memory buffers, causing the NPU to malfunction. The DNN model compiler can allocate the tensors to a shared on-chip memory space

instead of to separate designated buffers. Thus, sharing the on-chip memory space across the operator inputs, outputs, and weights can mitigate the problem observed with designated on-chip memory buffers. However, improper tensor allocation will overuse the shared memory space on the chip and disrupt the performance of DNN inference applications, causing the NPU to malfunction. Since DNN model parameters and inference executions are deterministic, our work creates a model compilation framework that resolves these two challenges, first by optimizing the inter-layer operator scheduling policy and second by improving the tensor allocation process and thus improving the energy efficiency and performance of the DNN inference applications on an NPU.

Prior Work: Conventionally, NPUs [29], [30] schedule DNN model operators in a layer-wise manner that first completes an entire operator before continuing to the next operator. The advantage of the layer-wise operator scheduler is that it only requires preserving the weight of one operator at a time in the on-chip memory space. However, because the intermediate tensors in a DNN model vary in size, larger intermediate tensors may cause the layer-wise operator scheduler to run out of on-chip memory space. The layer-wise operator scheduler also makes reusing the intermediate tensors difficult because saving the inputs and outputs of multiple operators is almost impossible with the on-chip memory space limitations. To demonstrate, Fig. 1 compares the on-chip memory hit rates of the inter-layer and layer-wise operator schedulers in our present work. Fig. 1 averages the on-chip memory hit rate of our DNN model benchmarks. In Fig. 1, the on-chip memory hit rates of the weights and the intermediate tensors using the inter-layer operator scheduler are respectively 0.5 and 0.94; by comparison, the on-chip memory hit rates of the weights and the intermediate tensors using the layer-wise operator scheduler are respectively 0.95 and 0.51. Previous research on data mappers [51], [59], [60] has aimed to identify the optimal data mapping of a single operator on a wide range of DNN accelerators. By contrast, our work focuses on multiple operators and how they can be adaptively scheduled to minimize off-chip memory data traffic.

Solutions: In the present work, we create a DNN model compilation framework called *EDA* that adjusts the operator traversal depth of the inter-layer operator scheduler as the on-chip memory in an NPU decreases in size. The operator traversal depth means the number of operators that the operator scheduler traverses. EDA allows for the on-chip memory hit rates of both weights (0.95) and intermediate tensors (0.96) to be maximized through an intelligent operator scheduling policy and on-chip shared memory management (see Fig. 1), in contrast to the inter-layer and layer-wise operator schedulers. First, EDA transforms a DNN model graph into a model that divides the tensors into multiple small tiles. Second, EDA divides the model graph into multiple subgraphs. Third, EDA ensures that the on-chip memory space is large enough to store the small tiles and the weight data of operators in each subgraph. Fourth, EDA employs offline profiling results to arrange the shared on-chip memory space in the presence of

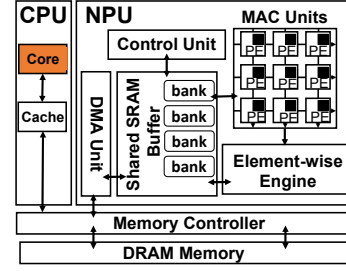


Fig. 2. The Neural Processing Unit (NPU) Architecture used in EDA

sparse tensor data. Finally, EDA resets the on-chip memory space after completing all operators in each subgraph. As a result, EDA improves the NPU energy efficiency and the performance of DNN inference applications by properly saving reused data from a DNN model in the on-chip memory and minimizing the amount of off-chip memory traffic. Our contributions can be summarized as follows.

- EDA adjusts the operator traversal depth of inter-layer operator scheduler to reduce off-chip memory traffic. EDA respectively achieves geometric means of $2.08\times$ and $2.39\times$ energy efficiency and performance speedup over the NPU with designated on-chip memory buffers.
- The EDA tensor-splitting model substantially minimizes on-chip memory space usage while removing unnecessary computation shown in cascading scheduling of ARM Vela model compiler [10].
- EDA manages the shared on-chip memory space of an NPU to improve the performance of DNN inference applications by unleashing the limitations of designated memory buffers in previous NPUs [29], [30].

II. BACKGROUND

A. Edge NPU Memory Hierarchy

The edge neural processing unit (NPU) often stores deep neural network (DNN) inference data on different types of memory. For example, Google Tensor TPU [63] usually includes LPDDR DRAM memory to preserve many weights and intermediate tensors of a DNN model. The mobile NPU also contains a small on-chip memory space to avoid continuously pulling data from the off-chip memory. Fig. 2 illustrates that the NPU in our present work contains the MAC unit, the element-wise engine, the shared SRAM memory space, and other control units. First, the NPU controller retrieves data from the off-chip memory space through the direct memory access (DMA) engine. Second, the NPU saves weights, inputs, outputs of a DNN model, and the lookup table (LUT) used in the element-wise engine in its on-chip memory buffer. The size of such a shared memory buffer often ranges from hundreds of KB to several MBs. In Fig. 2, the MAC unit is responsible for the operations of the convolution and fully-connected operators. In addition, the element-wise engine tackles other element-wise operators such as the ReLU. These compute units directly fetch and write back data from and to

the on-chip memory. The CPU tackles operators that the MAC unit and the element-wise engine do not support, such as the MEAN operator. The NPU usually takes hundreds of cycles to load data from the off-chip memory when the compute units cannot find the data within the on-chip memory space. The shared memory on the NPU chip is a direct-mapped cache and is programmable. Thus, there is no tag unit in the on-chip memory buffer. Furthermore, its arbitrator controls the crossbar, which distributes addresses to individual banks within the SRAM data array. Each bank inside the SRAM data array is 32-bit wide and can store two 16-bit data. In addition, an on-chip memory data array bank also has a decoder that allows for independent access to different rows in each bank. The latency to access the on-chip memory space is typically comparable to the register file access latency and is a single-cycle latency for the on-chip memory buffer lookup. The size of the memory buffer on the chip is usually proportional to the MAC unit size. However, the large on-chip memory also significantly increases the prices of the NPU. Hence, the present work intelligently allocates the on-chip SRAM memory space to minimize the number of off-chip memory transactions while reducing the peak on-chip memory usage.

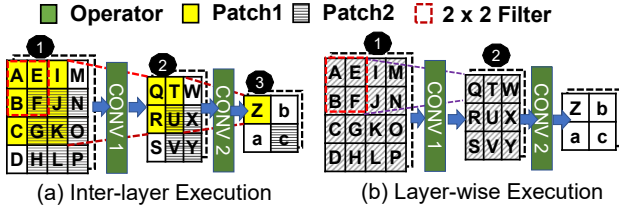


Fig. 3. The Inter-layer v.s. Layer-wise Scheduling

B. Inter-Layer Operator Scheduling

Fig. 3(a) illustrates the execution of an inter-layer operator scheduler in the DNN model compiler. As shown in Fig. 3(a), a 4×4 tensor is divided into multiple 3×3 patches. The CONV1 data patch is convoluted with a 2×2 filter, producing a 2×2 patch. The output patch of the CONV1 operator becomes the input of the CONV2 operator. Then, the CONV2 operator can directly use data stored in the on-chip memory rather than fetch data from the off-chip memory space. The inter-layer operator scheduler maximizes the intermediate tensor data reuse through deep operator traversal and stores whole weights of a DNN model in the memory space on the chip to further reduce the off-chip memory traffic. However, we observed that the small on-chip memory in an NPU is almost impossible to store all the weights of a DNN model. This limitation will significantly increase the off-chip memory traffic when using the inter-layer operator scheduler. In contrast, as shown in Fig. 3(b), the layer-wise operator scheduler that first finishes an operator (CONV1), yields a 3×3 output tensor and then continues to the next operator (CONV2). The layer-wise operator scheduler usually cannot reuse intermediate tensors of a DNN model well as their sizes

often alter. Hence, this work adjusts the traversal depth of the inter-layer operator scheduler to maximize the hit rate of the on-chip memory space in an NPU.

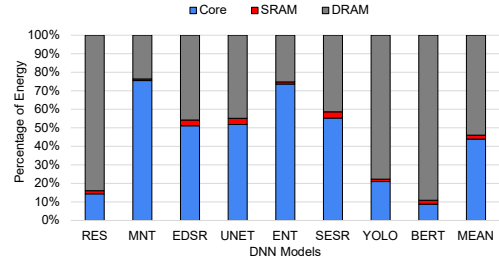


Fig. 4. Energy Consumption of the NPU with Layer-Wise Operator Scheduler

C. The NPU Energy Consumption

The MAC unit, off-chip memory, and on-chip memory constitute the crucial energy consumption of an NPU that includes designated on-chip memory buffers, 1MB in total. As shown in Fig. 4, the energy consumption of the memory system dominates the entire NPU across different DNN inference workloads. The NPU used in Fig. 4 schedules the operators in a layer-wise manner. The average percentage of energy consumption in the MAC unit, the on-chip memory, and the off-chip memory is 44%, 2%, and 54%, respectively. The technology node of the MAC unit on the NPU shown in Fig. 4 is 45 nm [19]. Jouppi et al. [29] indicate that the logic operation energy of the 7nm technology node is approximately $2.6 \times$ lower than the 45nm technology node. Therefore, after the MAC unit uses the 7nm technology node, the energy consumption of the NPU in the present work will become 27%, 3%, and 70%, respectively. In addition, in Fig. 4, the ResNet (RES) model [11] consumes 84% energy in the off-chip memory, because the RES model has large weights and intermediate tensors. The SESR [11] is designed for super-resolution image processing composed of tiny weights but large inputs and outputs. Consequently, the SESR off-chip memory energy takes 41% of the total energy consumption. The SESR on-chip memory also consumes 3% energy because the NPU frequently fetches data from the on-chip memory. In addition, the BERT [7] consists of multiple FC operators and the off-chip memory dominates the energy consumption after using the specialized MAC unit on the NPU. The evolution of the node technology can reduce the energy consumption of the MAC unit. However, the memory bandwidth and size on the edge NPU are limited and valuable. As a result, this work reschedules the execution order of operators to increase the data reuse on the on-chip memory and reduce the energy consumption of DNN inference workloads on the NPU.

III. EDA MODEL COMPILATION ARCHITECTURE

This section presents an overview of the EDA model compilation framework and its model transformation method to address problems of the inter-layer operator scheduler in a DNN model compiler.

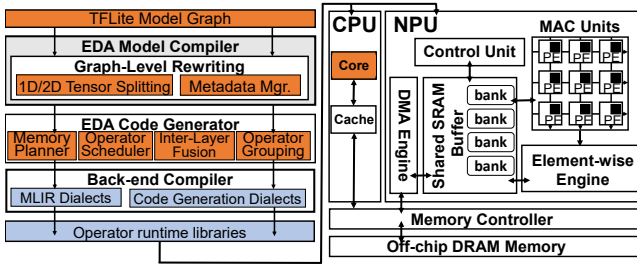


Fig. 5. EDA Model Compilation Framework

A. EDA Model Compilation Framework Overview

DNN model compilers [16], [18], [45], [57], [66] transparently schedule operators, fuse operators, and transform model graphs to improve the performance of DNN applications. Thus, the DNN model compiler simplifies the design of the NPU hardware and mitigates the complexity of the microarchitecture of the on-chip memory. For example, TensorFlow Lite for Micro (TFLM) [18] initially creates a static arena memory space corresponding to the memory space on the NPU chip at compile time. TFLM allocates tensor data in such an arena memory space, and then the NPU uses the results of this tensor allocation at compile time to manage its on-chip memory space during the runtime. EDA usually behaves as the TFLM and determines the tensor allocation in the on-chip memory of an NPU at compile time. Unlike TFLM, EDA changes the on-chip memory management policy by reusing the intermediate tensors of a DNN model.

As illustrated in Fig. 5, EDA consists of model graph-level rewriting and a code generator. The EDA model graph-level rewriting splits tensors of a DNN model into small tiles and properly inserts additional operators for its operator scheduler. The EDA code generator converts the JSON schema of a TFLite file into C/C++ codes, including the host and NPU operator codes. The EDA code generator outputs also describe the execution order of operators in a DNN model. The TFLite data format uses the JSON schema to depict the DNN model architecture and its associated parameters, such as operators and tensors. Initially, EDA translates a DNN model graph into a tensor-splitting model including small tiles of intermediate tensors. EDA changes the JSON schema of a TFLite file to represent the new tensor-splitting model. Note that the EDA tensor-splitting model does not alter the accuracy of the pretrained model. The number of MACs and the outputs of the tensor-splitting model are usually identical to the original model. Second, EDA employs its inter-layer fusion cost model to divide a DNN model graph into multiple subgraphs. Unlike previous inter-layer operator schedulers [9], [42], [62], EDA reduces the operator traversal depth in each subgraph of a DNN model. Third, the EDA code generator transforms the JSON schema of a TFLite file into C/C++ codes that include the host and NPU codes. Finally, the NPU back-end compiler translates outputs of the code generator into the object file and offloads operator kernels such as convolution,

depthwise convolution, and fully connected operations to the MAC unit of the NPU. The element-wise engine is responsible for complicated functions such as ReLU, Sigmoid, multiply, add operations, and so on. In summary, EDA changes the execution order of the operators and manages the memory space on the chip at compile time to decrease the off-chip memory traffic.

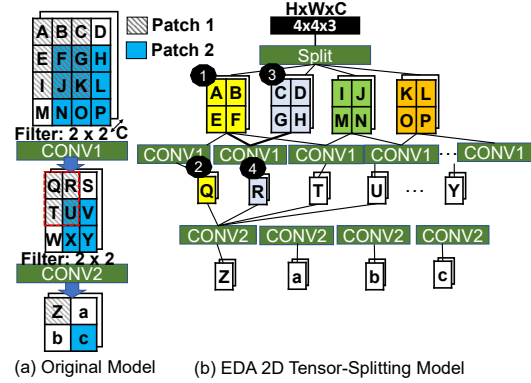


Fig. 6. EDA Tensor-splitting Model Transformation

B. 2D Tensor-Splitting CNN Model Transformation

Using small tiles of intermediate tensors in a DNN model decreases the memory usage in an NPU. The DNN model compiler [10], [42] usually breaks a tensor into multiple tiles and executes them to fit the small on-chip memory space of an NPU, as shown in Fig. 6(a). However, the cascade scheduler [10] in the DNN model compiler uses overlapping patches that expose redundant computation during CNN model inference. EDA eliminates such performance overhead by fetching tiled tensors as needed and eliminating duplicated calculations in the presence of overlapping patches. For instance, EDA will bypass the calculation to yield the data U in patch 2. In Fig. 6(b), EDA transforms the computation of each patch as a tensor-splitting model. At first, EDA divides a $4 \times 4 \times 3$ tensor into four $2 \times 2 \times 3$ tiles. Second, EDA uses the first $2 \times 2 \times 3$ tile of a tensor and a 2×2 filter to generate a $1 \times 1 \times 3$ output with the Q label through the CONV1. Third, EDA retrieves the first and second tiles of a tensor to produce an output of $1 \times 1 \times 3$ with the label R. Next, EDA continues to gain other $2 \times 2 \times 3$ tiles to produce a $2 \times 2 \times 3$ input patch of the CONV2. EDA employs these tiled tensors and operators to reduce the peak on-chip memory space usage while reusing intermediate tensors of operators in the memory space of the NPU chip. EDA divides tensors of convolution (CONV), depthwise convolution, fully-connected (FC), ReLU, and pooling operators into tiles. EDA stops its tensor-splitting operation in the presence of non-element-wise operators such as SOFTMAX and MEAN. EDA shares the metadata fields that are similar to previous TinyTS [45] work to decrease the amount of metadata.

C. Tensor-Splitting on Transformer Models

Natural language applications often use transformer models [22], [27] to generate texts and translate languages. Unlike CNN models, the transformer model uses significant numbers of FC operators multiplying matrices in the self-attention and feed-forward network (FFN) operations. As shown in Fig. 7(a), the self-attention consists of Query (Q), Key (K), and Value (V) data. Initially, the data of Q and K multiply with their weight data, and then an FC operator multiplies their results again before entering the Softmax operator. Next, the self-attention multiplies the Softmax operator's outputs with the V data. Then, their results will pass through multiple FC operators in the FFN processing. In a multi-head transformer model, the self-attention and FFN usually iterate multiple times. Unlike the CONV operator, the FC operator has no filter that performs the dot product with each element of the input feature map. As shown in Fig. 7(b), the FC operator uses the inner product that multiplies a row of the matrix A with a column of the matrix B. To meet the EDA 2D tile form, EDA breaks two matrices into multiple small tiles according to the blocked matrix multiplication. The on-chip memory of an NPU saves each tile of matrices.

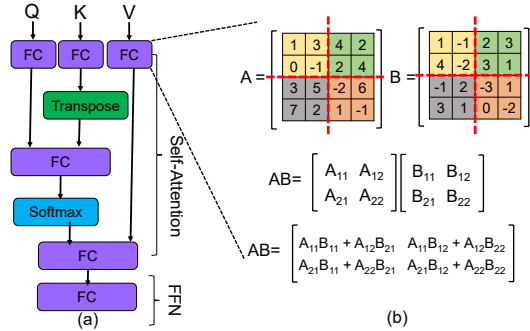


Fig. 7. Transforming the FC Operator into the Blocked Matrix Multiplication

The execution order of FC operators in each tiled tensor determines the data reuse rate of the on-chip memory in an NPU. A transformer model consists of several FC operators that connect with each other, as shown in Fig. 8(a). EDA converts a model graph of Fig. 8(a) into a new one, as shown in Fig. 8(b), where an input matrix consists of multiple tiles with its associated FC and ADD operators. One of the approaches to reuse tiled outputs of the FC1 operator is to pass the partial sum ($A_{11}B_{11}$) to the FC2 operator. Typically, the FC2 operator must prepare for a 16-bit register or on-chip memory space to save the partial sum ($A_{11}B_{11}$) of the FC1 operator when each element in a matrix takes 8 bits. Then, the FC2 operator can continue to take the partial sum of the FC1 operator but must store its output in a 32-bit register or memory space. Such a value propagation will amplify the storage space requirement to save the output of each FC operator.

Instead of passing partial sums to the next FC operator, EDA first completes the $A_{11}B_{11} + A_{12}B_{21}$, quantizes this value, and passes such a value to the FC2 operator. Another option

is for EDA to continue to compute the $A_{11}B_{12} + A_{12}B_{22}$ after completing $A_{11}B_{11} + A_{12}B_{21}$ to reuse the A_{11} and A_{12} stored in the on-chip memory of an NPU. EDA usually breaks a DNN model into multiple subgraphs to reduce the length of its operator traversal. Thus, EDA creates two subgraphs in a subset of a transformer model as shown in Fig. 7(a). In Fig. 7(a), the first subgraph includes FC operators associated with the data of Q and K. The second subgraph contains FC operators of the V data and the FFN processing. Each tiled tensor K fuses the transpose and FC operator. EDA reuses tiled tensors of FC operators in a subgraph to reduce the off-chip memory traffic.

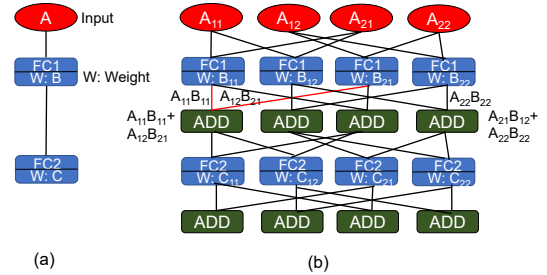


Fig. 8. Tensor-Splitting on the FC operator

IV. EDA HARDWARE-AWARE OPERATOR SCHEDULING

Maximizing the data reuse rate of operator's intermediate tensors on the small memory space in the NPU is challenging, particularly because the size of tensors is different across operators in a DNN model. This section describes the EDA inter-layer fusion cost model that adjusts the operator traversal depth of each subgraph and manages the shared memory space on the NPU. In addition, a small tiled tensor will cause under-utilization of the MAC unit in an NPU. Hence, this section also addresses the EDA operator grouping technique that improves the utilization of the MAC unit.

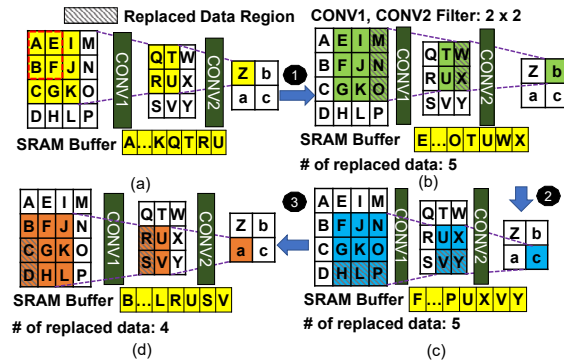


Fig. 9. EDA Inter-layer Operator Traversal

A. EDA Inter-layer Operator Scheduling

The inter-layer operator scheduler minimizes off-chip memory transactions by reusing tiled intermediate tensors of op-

erators in stacked DNN models. In addition, the inter-layer operator scheduler optimistically assumes that the on-chip memory can preserve tiled tensors and weights of multiple operators. The inter-layer operator scheduler completes a part of the tensor data in each operator at a time. Therefore, it can reuse the output of the CONV1 operator stored in the on-chip memory space of an NPU, as shown in Fig. 9. EDA usually gathers multiple tiled tensors in a data patch. EDA refers to the output size of the last operator in a subgraph and the filter size of the operators to infer the size of a data patch in each operator. For example, in Fig. 9, EDA finally yields a CONV2 output and the filter size of both operators is 2×2 . Thus, the CONV1 and CONV2 operator's input patch size is 3×3 and 2×2 , respectively. Suppose that the SRAM buffer can hold nine inputs and four outputs. Since CONV2 is the last operator, the NPU writes the output of CONV2 directly to the DRAM memory. In Fig. 9(a), firstly, EDA produces four outputs of the CONV1 operator. Then, EDA uses these four outputs and a 2×2 filter to generate an output of the CONV2 operator. Second, EDA replaces the A, B and C data with the M, N and O data and also re-expands the Q and R since the size of the SRAM buffer is limited. The amount of replaced data counts the data that the NPU places in the on-chip memory. Thus, the total number of replaced data is 5, as shown in Fig. 9(b). Third, Fig. 9(c) performs similarly to Fig. 9(b) and results in 5 replaced data. Note that, in Fig. 9(d), EDA only needs to fetch the data C and D from the DRAM memory to produce the data S because the data R has been created in Fig. 9(a). In addition, EDA needs to fetch the R data to generate the output a of the CONV2 operator. Therefore, the total amount of replaced data in Fig. 9(d) is 4. Finally, the total number of replaced data in Fig. 9 is 14. In Fig. 9, the inter-layer scheduler reuses the CONV1's outputs stored in the on-chip SRAM memory buffer to reduce the off-chip memory data traffic.

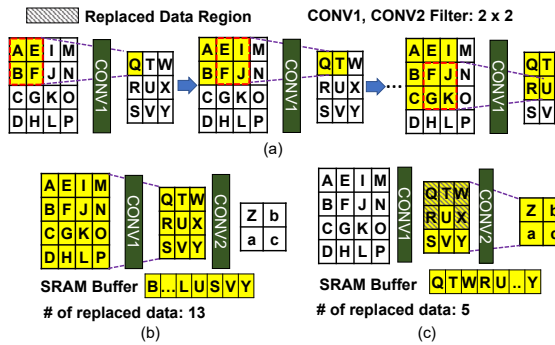


Fig. 10. Layer-wise Operator Traversal

As shown in Fig. 10, the layer-wise operator scheduler yields whole outputs of the CONV1 operator at first and uses these outputs to be the input of the CONV2 operator. Fig. 10(a) illustrates a 2×2 filter that loops through the 4×4 inputs of the CONV1 operator. The next two runs of the layer-wise operator are similar to Fig. 9(b) and Fig. 9(c) and

the amount of the replaced data is 10. However, in the last run of Fig. 10(b), the amount of replaced data is 3, because the layer-wise operator scheduler only needs to yield the data S and fetches the data C and D into the on-chip memory. Thus, the layer-wise operator scheduler results in 13 replaced data in completing the CONV1 operator's outputs. Finally, the layer-wise operator scheduler also loads five CONV1 outputs in the on-chip memory to produce the CONV2 outputs. Hence, in total, the amount of replaced data of the layer-wise operator scheduler is 18. Unlike EDA, the layer-wise operator scheduler increases the number of off-chip memory data traffic when fetching the replaced CONV1 data back to the on-chip memory to yield outputs of the CONV2.

B. Creating Tensor-splitting Model Subgraphs

The prior inter-layer operator scheduler [9] assumes the on-chip memory can store the entire weight data in a DNN model. However, the size of weights increases as the growth of the channel, filter count of the weight data, and the operator count in a DNN model. The frequent weight data movement across different memory hierarchies becomes a critical bottleneck of the inter-layer operator scheduler. Hence, EDA breaks down a DNN model graph into multiple subgraphs and decreases off-chip memory traffic by reducing the on-chip memory usage of an NPU to store the weight data in a subgraph.

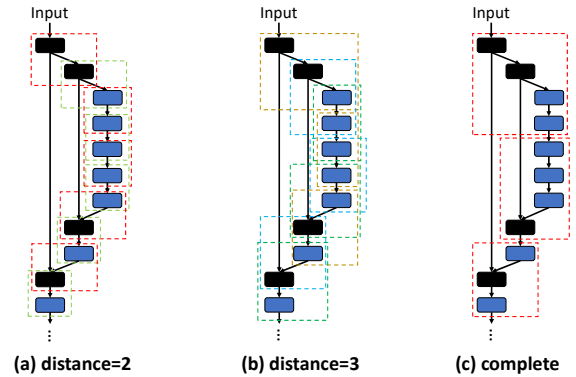


Fig. 11. Searching Depth of EDA Traversal Path in a Subgraph

EDA guarantees that operators in a subgraph do not run out of the memory space in an NPU while minimizing the off-chip memory traffic. However, the number of operator clustering scenarios grows exponentially with the increase of operator counts in a DNN model. To decrease the operator clustering time when building a subgraph, EDA reduces such subgraph building as the matrix chaining problem [13] and solves this problem using the dynamic programming technique. Firstly, as illustrated in Fig. 11, EDA examines the combination of operators with different distances. For example, EDA starts to cluster two and three operators in the beginning as a subgraph in Fig. 11(a) and Fig. 11(b). Second, EDA records the distance of each operator combination that does not run out of the on-chip memory space and marks a root operator

of each potential subgraph. Third, EDA employs its inter-layer fusion cost function to determine the data movement cost of each potential graph. Fourth, EDA compares the cost number of each potential subgraph with the same root operator. Finally, EDA decides the depth of the subgraph, which has the minimum cost number, as shown in Fig. 11(c). In addition, EDA decides the operator traversal depth of a subgraph based on the size of the memory space on the chip. The amount of input sparsity also affects the usage of the on-chip memory space. Thus, EDA considers the input sparsity shown in a DNN model when building subgraphs. Typically, the amount of sparse input data varies between different inputs of operators. However, our experiments reveal that the input sparsity of operators does not significantly fluctuate even when changing input data of a DNN model. Hence, EDA uses offline profiling to estimate the requirement of memory space in an NPU and reflects on-chip memory usage in the presence of the input sparsity shown in a DNN model when building subgraphs.

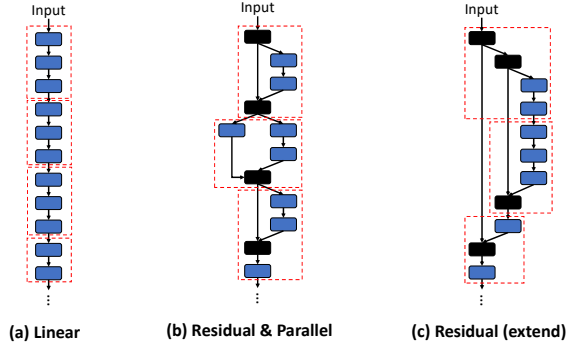


Fig. 12. Creating EDA Model Subgraphs in Different Model Architectures

The operator's data dependency also determines the operator's lifetime and the depth of a subgraph. As illustrated in Fig. 12, EDA summarizes three types of stacked CNN model architectures. Fig. 12(a) presents a simple linear stacked model where the degree of data dependency in each operator is 1. The YOLOv2 [5] model in our benchmark belongs to this linear stacked model. Since the degree of data dependency of this linear stacked model is straightforward, EDA directly determines the depth of each subgraph based on the tensor and the weight size of an operator. In addition, Fig. 12(b) shows a residual block model architecture where the black operator has two data dependencies and two parallel operators are in a layer. Since EDA ensures its operator traversal that covers all operators, EDA will particularly record the information of parallel operators in the convergence operator as the black node shown in Fig. 12(b) when searching for the depth of the subgraph. EDA relies on the convergence operator to recognize the existence of parallel operators. Thus, a subgraph is required to contain at least one convergence operator that helps EDA to know the existence of parallel operators. The MobileNet [11], ResNet [11] and EfficientNet [11] models in our benchmark belong to this model type, as shown in Fig. 12(b). Next, an operator might have a long list of dependent operators

and finally converge on its sibling operator, as shown in Fig. 12(c). EDA might divide these dependent operators into multiple subgraphs. In this case, EDA will also add a mark in the convergent operator to ensure the correct number of memory transactions used during the operator scheduling. The EDSR [4], SESR [11], and UNet [3] models belong to this type of model in the presence of Fig. 12(c).

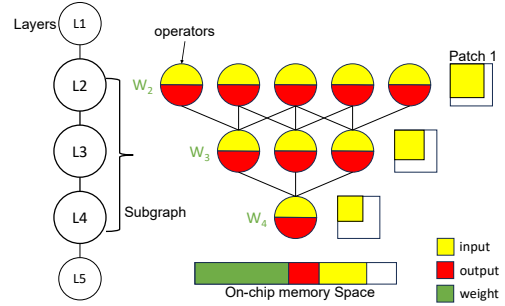


Fig. 13. EDA FUSION Operation

C. Inter-Layer Fusion Cost Model

The EDA inter-layer fusion cost model decides the operator traversal depth of each subgraph in a tensor-splitting model. To reduce peak on-chip memory usage, EDA first completes partial data of an operator and then continues to the next operator. Thus, in a tensor-splitting model as shown in Fig. 13, EDA only considers the storage requirement of each operator's tensor data patch in its inter-layer fusion cost model. As shown in Fig. 13, EDA also ensures intermediate tensors and weights in a subgraph including three layers do not overuse the memory space on the NPU chip. Firstly, EDA uses dynamic programming to enumerate possible subgraphs of a tensor-splitting model. Next, as illustrated in Fig. 13, the EDA FUSION function calculates the on-chip memory space requirement to store inputs, outputs, and weights within data patches of each operator in a subgraph. EDA keeps track of valid subgraph combinations, where operators do not overuse the memory space in an NPU. Then, EDA measures the cost of a subgraph that indicates the amount of off-chip memory data movement. Finally, EDA examines the cost of subgraphs and combines subgraphs to maximize the reuse of operator outputs on the memory space of the NPU chip.

In Algorithm 1, the 2D array F in line 2 maintains an operator in each layer of a DNN model. When a layer includes multiple operators as the residual block in the ResNet, EDA will use different elements in the 2D array F to keep track of these operators. EDA loops through all possibilities to decide the length of each subgraph in a tensor-splitting. The loop in line 8 scans through all possible depths of a subgraph. Then, the loop in line 9 tackles the marking of a root operator in each potential subgraph. At last, the loop in line 12 results in the cost number and determines the length of each subgraph through the FUSION function. The FUSION procedure from line 19 to line 29 in Algorithm 1 decides whether to add a

Algorithm 1 EDA Layer Fusion Cost Model Algorithm

Input: $L_1, L_2, \dots, L_n$ - layers in the model; Input, output cost of layer L_i is L_i^i and L_i^o

Output: The optimal fuse state F for all layers in the model

```

1:  $B \leftarrow$  make an empty structure to hold layers' each buffer size.
2:  $F \leftarrow$  make an empty structure to hold the layers' indices.
3:  $cost \leftarrow$  make an empty structure to hold the costs.
4: for  $i \leftarrow 1, 2, \dots, n$  do
5:    $F[i, i] \leftarrow [[L_i]]$ 
6:    $cost[i, i] \leftarrow L_i^i + L_i^o$ 
7: end for
8: for  $d \leftarrow 1, 2, \dots, n - 1$  do
9:   for  $i \leftarrow 1, 2, \dots, n - d$  do
10:     $j \leftarrow i + d$ 
11:     $cost[i, j] \leftarrow$  maximum value
12:    for  $k \leftarrow i, i + 1, \dots, j - 1$  do
13:       $cost[i, j] \leftarrow \min(cost[i, j],$ 
        FUSION( $i, k, j, cost, F$ ))
14:       $F[i, j], B[i, j] \leftarrow \arg \min(cost[i, j],$ 
        FUSION( $i, k, j, cost, F$ ))
15:    end for
16:  end for
17: end for
18: return  $F[1, n], B[1, n]$ 

19: function FUSION( $i, k, j, cost, F$ )
20:    $U \leftarrow$  last element in  $F[i, k]$   $\triangleright$  single layer or fusion layers
21:    $L \leftarrow$  first element in  $F[k + 1, j]$   $\triangleright$  single layer or fusion layers
22:   if  $U$  and  $L$  can be fused together then
23:      $Fused \leftarrow cost[i, k] + cost[k + 1, j] - L_k^o - L_{k+1}^i$ 
24:      $Separated \leftarrow cost[i, k] + cost[k + 1, j]$ 
25:     return  $\min(Fused, Separated)$ 
26:   else
27:     return  $cost[i, k] + cost[k + 1, j]$ 
28:   end if
29: end function

```

new operator in a subgraph. For instance, assuming that the content of the 2D array F is $[[0, 1, 2], [3], [4, 5]]$, which indicates three subgraphs, and each subgraph has 3, 1, and 2 operators. Furthermore, the U and L in line 20 and line 21 mean the subgraphs $[0, 1, 2]$ and $[3]$ in the 2D array F . Then, the FUSION function will decide whether to move the subgraph with the operator 3 into the first subgraph. EDA examines operators in each layer, as illustrated in line 8 to line 17 of Algorithm 1, to update the operator traversal depth of each subgraph. At last, EDA uses the buffer variable to keep track of the on-chip memory usage in each subgraph and ensures operators in a subgraph do not overuse the memory space on the chip.

D. On-Chip Shared Memory Allocation

The baseline NPU contains a shared on-chip memory that stores the inputs, outputs, and weights of a DNN model. Previously, the NPU [30] statically partitioned the on-chip memory space, and each memory partition was tailored to the layer-wise operator scheduling. For instance, the size of the weight buffer in the prior NPU [30] is tiny because the layer-wise operator scheduler only needs to keep the weight data of an operator in the memory space on the chip. However, such a memory allocation will significantly increase the off-chip memory data traffic since EDA often traverses across multiple operators. Hence, the statically partitioned memory allocation also limits the performance of EDA. Typically, the shared on-chip memory improves the flexibility of the memory allocation. However, EDA will raise out-of-memory exceptions without properly managing the shared on-chip memory of the NPU.

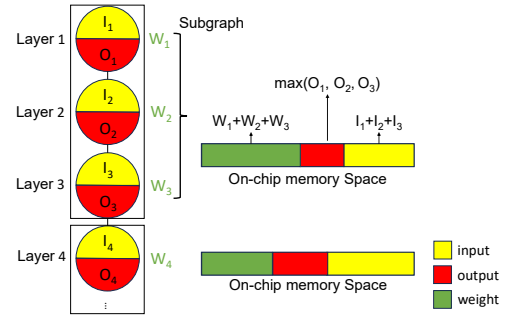


Fig. 14. EDA On-Chip Shared Memory Allocation

EDA creates a static arena memory buffer whose size is the same as the on-chip memory space in an NPU. EDA also considers double buffering in its memory allocation. Firstly, as illustrated in Fig. 14, EDA places a subgraph's weight data and inputs in the on-chip memory. Note that the input here is only the size of inputs in a data patch. The weight data here gathers weights of three layers in a DNN model. Furthermore, EDA reuses operator outputs in a subgraph, and the size of these outputs differs across operators. Therefore, EDA only allocates memory space for the largest outputs in a subgraph to fit all outputs. EDA also relies on its offline profiling results to complete such a memory allocation in the presence of the input sparsity in a DNN model. In addition, EDA will clean the on-chip memory space when completing a subgraph to ensure the correctness of the memory allocation. Unlike the static memory partition, EDA also alters the size of each on-chip memory partition with the change of tensor sizes in each subgraph, as shown in Fig. 14. At last, EDA examines potential subgraphs and ensures operators in these subgraphs do not run out of on-chip shared memory.

E. Operator Grouping

In the EDA tensor-splitting model, EDA breaks operators and tensors of a DNN model layer into multiple tiles. The

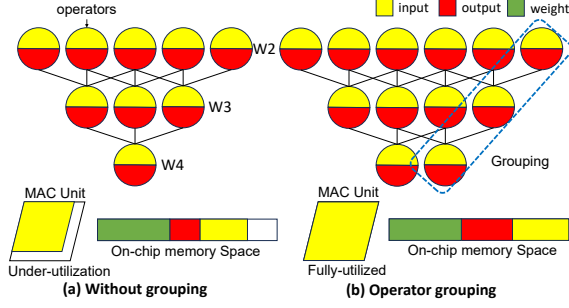


Fig. 15. EDA Operator Grouping

size of tiled tensors can be different. EDA usually groups one or multiple tiles as a data patch to yield an output of an operator. However, these fine-grained tiled tensors in a data patch sometimes do not fully utilize the entire MAC unit and the memory space on the NPU chip. To resolve such a problem, EDA operator grouping will gather more tiled tensors and data patches to improve the utilization of the MAC unit and the on-chip memory. The EDA operator grouping occurs after its inter-layer fusion cost model. The EDA operator grouping fuses more tiled operators and data patches in a model layer when the MAC unit and the on-chip memory space of an NPU have not been fully utilized. Fig. 15(a) illustrates a data patch in a subgraph composed of three layers. Firstly, EDA places tiled tensor segments in a data patch in the on-chip memory space. However, EDA finds such a data patch does not fully utilize the MAC unit and on-chip memory. Thus, EDA continues to merge more tensor fragments into a data patch to improve the hardware utilization shown in Fig. 15(b).

TABLE I
EDA NPU SPECIFICATION

Number of MAC PEs	128 × 128
Technology of PEs	45 nm
PE Clock Frequency	700 MHz
128 × 128 PE Energy	4447.43 pJ/cycle
On-chip Shared Buffer Size	1MB
Off-Chip Memory Bandwidth	25.6 GB/s
Off-Chip Memory Clock Frequency	1.6 GHz
Off-Chip Memory Energy	12 pJ/bit

V. METHODOLOGY

A. Hardware Modeling

To evaluate EDA on different hardware parameters of an NPU, we built our EDA model compilation framework, and DNN model benchmarks on our cycle-accurate systolic array simulator derived from Planaria [1]. Table I presents the NPU hardware parameters used in our NPU simulator. Our NPU contains a systolic array using the weight-stationary dataflow where the size of the MAC unit is 128 × 128. We synthesize such a systolic array with the Synopsys Design Compiler using the FreePDK 45nm standard cell library [19] to extract

the energy consumption and latency of the MAC unit. Our MAC unit includes 16-bit registers, and its clock frequency is 700 MHz. We also employ the IOSESC [62] sparse MAC unit to eliminate the computation of input sparse data using Compressed Sparse Fiber (CSF) compressed data format. EDA automatically fuses the convolution and activation function operators, such as ReLU and LeakyReLU. In a fused operator, the MAC unit first stores an output element in the shared memory space on the NPU. Then, the element-wise engine gains such an output from the on-chip memory space, and the LUT completes the activation function of such an element in a cycle. The reconfigurable LUT holds 512 16-bit values that are directly mapped to the shared memory space on the NPU. The off-chip LDDR4 memory bandwidth is 25.6 GB/s [35], and its energy consumption is 12 pJ/bit [40]. In addition, we use CACTI [48] to estimate the energy of the on-chip memory, which is composed of double buffering to hide off-chip memory access latency. A double buffer contains a working set and an idle set. At a given cycle, the data in the working set feed the MAC unit, while the idle set fetches data from the off-chip memory.

TABLE II
EDA DNN MODEL BENCHMARKS

	Input Size	Weights(MBs)	Input Sparsity (%)
ResNet(RES) [11]	224x224x3	44.78	21.7%
MobileNet(MNT) [11]	224x224x3	6.62	36.8%
Edsr(EDSR) [4]	224x224x3	2.89	34.8%
Unet(UNET) [3]	224x224x3	3.74	45.3%
Efficient(ENT) [11]	224x224x3	8.79	37.5%
Sesr(SESr) [11]	1920x1080x1	0.03	50.6%
Yolov2(YOLO) [5]	224x224x3	21.03	0.9%
MobileBERT(BERT) [7]	1x384x512	0.88	0%

B. Edge DNN Benchmarks

Table II presents model parameters of each DNN model in the EDA DNN benchmark. Each DNN model has different model parameter values, and its weight data is not pruned. In Table II, SESr [11] generates a high-resolution image from a low-resolution input. The SESr model [11] takes 1080p input and outputs 4K images. Furthermore, the EDSR [4] is used for the Single Image Super-Resolution application. These DNN models shown in Table II are composed of convolution, depth-wise convolution, fully connected, and other operators. Multiple DNN models in Table II include the ReLU and ReLU6 operators that often produce sparse data. We observed the input sparsity ratio of a DNN model does not significantly fluctuate with the change of inputs in a dataset. Thus, EDA refers to the estimation of its off-chip profiling to allocate tensors on the memory space of an NPU in the presence of input sparse tensors. In Table II, EDA averages the percentage of sparse data shown in each intermediate tensor of a DNN model. In Table II, the percentage of tensor sparsity in the YOLO model is only 0.9%, because it instead uses the LeakyReLU operator. The size of weights also varies across different DNN models. For example, the ResNet [11] and Yolov2 [5] models have a large weight, but the weight size

on the SESR [11] is only about 30KB, as shown in Table II. MobileBERT is a transformer model including multiple FC operators. We evaluate the subset of a MobileBERT model that contains a self-attention and a FFN process. MobileBERT does not contain any input sparse tensor, since it does not use the ReLU operator.

C. Operator Schedulers

In the present work, we compare the energy and the performance variation of different inter-layer operator schedulers such as FUSED [9], IOSESC [62] and EDA. FUSED [9] behaves as the cascade operator scheduler [10] that first traverses through the initial tiled tensor of all operators in a DNN model and then jumps to the next subset of tiled tensors. IOSESC [62] also employs the inter-layer operator scheduling policy but eliminates the computation on the sparse input tensors. LAY [30] uses a layer-wise operator scheduling policy. We deploy FUSED, IOSESC, and LAY operator schedulers in an NPU composed of designated on-chip memory buffers. Both input and output on-chip memory buffers are respectively 384KB, and the size of the weight on-chip memory buffer is 256KB. EDA-NS and EDA schedule operators in a subgraph of a DNN model with a shared on-chip memory space in the NPU. EDA-NS does not eliminate the computation of input sparse tensors. EDA-Static employs the EDA operator scheduling policy in the NPU composed of designated on-chip memory buffers. The size of each tiled tensor in the present work is empirically set to 1×32 .

TABLE III
ENERGY CONSUMPTION OF DIFFERENT OPERATOR SCHEDULERS (μJ)

	FUSED [9]	IOSESC [62]	LAY [30]	EDA-NS	EDA
RES [11]	51.1	50.0	12.3	8.5	7.5
MNT [11]	15.0	11.5	14.1	11.4	7.9
EDSR [4]	922.7	803.4	206.6	154.7	124.5
UNET [3]	17.4	14.5	7.9	4.7	2.8
ENT [11]	17.3	11.6	14.5	11.7	6.6
SESR [11]	580.9	264.4	250.3	168.8	91.3
YOLO [5]	16.3	16.3	3.3	2.9	2.8
BERT [7]	0	0	0.7	0.2	0.2

VI. EVALUATION

A. Energy Consumption of Operator Schedulers

Table III presents the energy consumption of FUSED [9], IOSESC [62], LAY [30], EDA-NS, and EDA in different DNN models. The average energy consumption of FUSED [9], IOSESC [62], LAY [30], EDA-NS, and EDA is $231.5 \mu\text{J}$, $167.4 \mu\text{J}$, $72.7 \mu\text{J}$, $51.8 \mu\text{J}$, and $34.8 \mu\text{J}$, respectively without counting the BERT model. FUSED [9] and IOSESC [62] usually employs patch-based inference [42] that divides the input feature map of the CONV operator into multiple overlapping patches. However, the BERT model includes multiple FC operators without having overlapping patches. Thus, FUSED [9] and IOSESC [62] cannot work on the FC operator. EDA and EDA-NS do not have such a limitation. As a result, they can both work on the FC operator and on the BERT model. In Table III, the energy level in the BERT model remains the same when

using EDA and EDA-NS. The tensor data size of most DNN models in Table III is larger than the on-chip memory space of an NPU. IOSESC [62] only saves non-zero tensor data on the on-chip memory space. Thus, the SESR model consumes less energy, since IOSESC [62] obtains more on-chip memory space to reuse data over FUSED [9]. EDA also obtains such an advantage when the sparse data is removed from the on-chip memory space of an NPU. In Table III, LAY consumes less energy than FUSED [9] and IOSESC [62], because LAY only needs to store the weight data of an operator in the on-chip memory space of an NPU. Together, reusing the weights of a DNN model in the on-chip memory space can significantly lower the off-chip memory traffic and therefore decrease the energy consumption of an NPU.

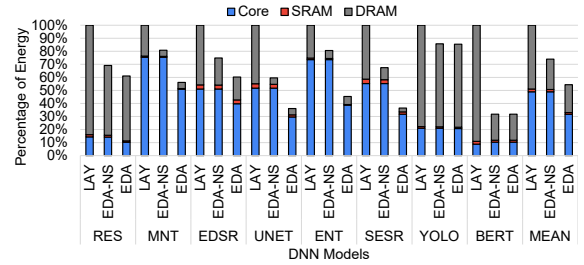


Fig. 16. Energy Reduction on Different NPU Memory Architectures

B. Energy Reduction of Different NPU Memory Architectures

Fig. 16 compares the MAC unit, on-chip SRAM memory, and off-chip DRAM memory energy reduction when using LAY, EDA-NS, and EDA on the NPU with different on-chip memory architectures. LAY allocates intermediate tensors of operators and weights of a DNN model on designated memory buffers. Unlike LAY, EDA and EDA-NS work on the NPU with shared on-chip memory space. As shown in Fig. 16, the energy levels of the MAC unit on EDA-NS and LAY typically remain similar because the number of MACs and outputs on EDA-NS and LAY are identical. Thus, EDA-NS reduces the 25% off-chip memory energy over LAY, since EDA-NS reuses the intermediate tensors and weights of a DNN model in the on-chip memory space. EDA also consumes 46% less energy on an NPU than using LAY, because EDA off-chip memory energy is 27% lower than LAY by reusing more data in the on-chip memory space. EDA also reduces the 17% energy of the MAC unit over LAY, since EDA eliminates the computation of sparse data. As a result, the NPU composed of the shared on-chip memory space uses less energy when using EDA and EDA-NS over LAY.

C. Performance of Operator Schedulers

Fig. 17 presents the normalized performance speedup of LAY, EDA-NS, and EDA in different DNN models. EDA-NS and EDA achieve geometric means of $1.83\times$ and $2.39\times$ speedup over the LAY operator scheduler on the NPU composed of designated memory buffers. In Fig. 17, using EDA on the BERT model results in $3.92\times$ faster than LAY, since the

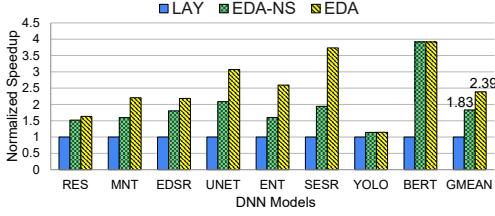


Fig. 17. Performance Comparison with Layer-Wise Operator Scheduler

BERT model consists of multiple memory-bound FC operators. EDA stores weights of operators and reuses intermediate tensors of operators in a BERT model subgraph on the on-chip memory space of an NPU to decrease the off-chip memory traffic. The BERT model also includes multiple non-element-wise operators that stop EDA from adding more FC operators in a subgraph. These non-element-wise operators reduce the number of operators and the amount of weight data in each subgraph. Thus, reusing intermediate tensors of operators and weights in the BERT model on the on-chip memory space of an NPU becomes possible and improves the performance of EDA and EDA-NS. In addition, EDA only preserves the nonzero input tensor data on the on-chip memory space of an NPU. EDA provides more on-chip memory space of an NPU than EDA-NS to reuse data of a DNN model. Hence, EDA outperforms EDA-NS by reducing off-chip memory traffic.

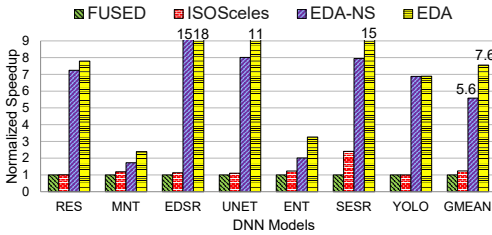


Fig. 18. Performance Comparison with Inter-Layer Operator Schedulers

Fig. 18 presents the normalized performance speedup of FUSED [9], IOSESC [62], EDA-NS, and EDA in different CNN models. IOSESC [62], EDA-NS, and EDA respectively achieve geometric means of $1.23\times$, $5.6\times$, and $7.6\times$ speedup over FUSE operator scheduler [9]. The operator schedulers in Fig. 18 execute the operators using inter-layer operator scheduling policy and reuse intermediate tensors of the operators. However, FUSED [9] and IOSESC [62] frequently pull weights of operators from the off-chip memory when jumping back and forth between different operators. Unlike FUSED [9], IOSESC [62] provides more on-chip memory space to store intermediate tensors of operators. Hence, using IOSESC [62] on the SESR model is $2.4\times$ faster than FUSED [9] even though both operator schedulers can hold weights of the SESR model in the on-chip memory space of the NPU.

D. Operator Schedulers on Different On-Chip Memory Sizes

Fig. 19 presents the normalized performance speedup of IOSESC [62] and EDA over LAY with the small on-chip memory space when altering the size of the on-chip memory space of an NPU. Both IOSESC [62] and EDA remove the input sparse data on the on-chip memory space of an NPU. EDA gains geometric means of $2.39\times$ and $2.68\times$ speedup on the small and large on-chip memory space of an NPU over LAY, respectively. The size of the small and large on-chip memory is 1MB and 4MB. IOSESC [62] works on the NPU composed of three designated memory buffers. The memory buffers of the input, output, and weight on the large on-chip memory are 1.5MB, 1.5MB, and 1MB. The size of each memory buffer on the small on-chip memory is 384KB, 384KB, and 256KB. Increasing the memory space on the chip can store more intermediate tensors and weights of a DNN model and also affects the performance of DNN inference. Thus, SESR, EDSR, UNET, ENT models gain $5.28\times$, $2.33\times$, $1.69\times$, $1.06\times$ speedup on IOSESC [62] when increasing the size of on-chip memory space. As shown in Fig. 19, the SESR and BERT model in IOSESC [62] also gain performance improvement by increasing the on-chip memory space although its weight buffer has already preserved the entire weight data of the SESR and BERT model. The size of the intermediate tensors in the SESR [11] model is huge and its performance is more sensitive to the size of the input and output on-chip memory buffer. Thus, IOSESC [62] improves the performance of the SESR model when increasing the on-chip memory size to store more intermediate tensors. In contrast, the performance of the MNT, ENT, and YOLO models on EDA does not fluctuate significantly when increasing the size of the memory space on the chip, because EDA has maximized the data reuse of these models when the size of the memory space on the chip is small. As a result, some DNN models on EDA can still gain performance speedup as the on-chip memory size drops.

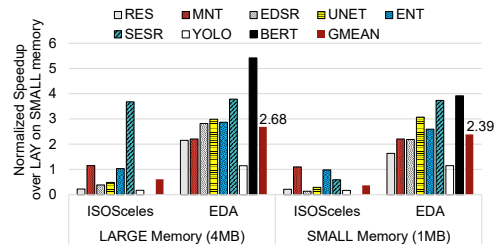


Fig. 19. Performance of IOSESC and EDA Operator Schedulers when using Different On-Chip Memory Sizes

E. EDA Performance on Different NPU Architectures

Fig. 20 compares the performance of EDA-static and EDA on an NPU with designated on-chip memory buffers or the shared on-chip memory space. The size of the shared on-chip memory space is 1MB. The size of the input, output, weight on-chip memory buffer is 384KB, 384KB, and 256KB on the NPU having designated memory buffers. EDA-static

and EDA achieve a geometric mean of $2.11\times$ and $2.39\times$ speedup over the LAY operator scheduler, respectively. EDA-static employs the EDA inter-layer operator scheduling policy on the NPU composed of designated on-chip memory buffers. Unlike EDA, EDA-static limits the on-chip memory space to adapt to changes in intermediate tensor and weight sizes. EDA-static and EDA gain $3.21\times$ and $3.73\times$ speedup over LAY in the SESR model, since EDA can obtain more on-chip memory space to save intermediate tensors of operators in the SESR model. Furthermore, EDA has more on-chip memory space to preserve weights of the BERT model, and thus EDA outperforms EDA-static.

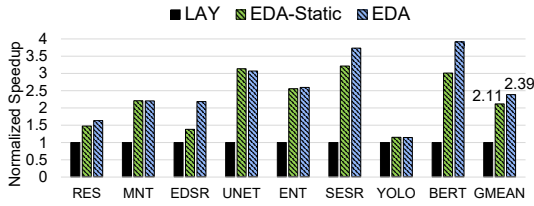


Fig. 20. EDA Performance Variations on Different NPU Architectures

F. EDA Compilation Overhead

Table IV presents the compilation time (seconds) of each EDA operation stage. The Table IV results were collected from the physical AMD Ryzen Treadripper 3.8GHz CPU [2]. EDA reads model parameters at the GRAPHING stage. In addition, EDA schedules operators in each subgraph at the SUBGRAPH stage in Table IV. EDA also groups operators. Then, the GROUP and SCHED stages finalize the execution order of operators. EDA spends about 10 seconds in the GROUP stage of UNET [3] model, since the UNET [3] has large inputs and many complex operators. Furthermore, EDA decreases the search space of SUBGRAPH from $O(2^n)$ to $O(n^4)$ where n is the number of operators or layers using the dynamic programming algorithm. Hence, the time of SUBGRAPH is proportional to the number of layers in a DNN model. In Table IV, the average total EDA compilation time of EDA is 7.17 seconds. The compilation time of the BERT model is about 2 seconds because the BERT model only contains one self-attention and FFN process.

VII. RELATED WORK

Tensor Mapping and Scheduling: Mapping emerging and complex DNN model data and applications to the rigid NPU while fully utilizing the hardware resources is challenging. The prior work [6], [20], [37], [46], [61], [64], [67], [68] invents novel dataflows to help NPUs adapt to the changes in DNN model architectures. For instance, AMOS [68], Tangram [20], MAERI [37], and FlexFlow [46] develop flexible dataflows to facilitate the reconfigurability of NPUs. In addition, FLAT [32] designs a novel dataflow to mitigate the attention model bottleneck. The CoSA [26], Simba [58], Planaria [21], and TSP [6] schedule multi-tenant applications in Chiplet and distributed spatial accelerators. ISOSceles [62] and other prior

work [9], [14] discuss the usage of inter-layer scheduling to eliminate the sparsity and explore the design space on the chiplet architecture. MAGIS's fission transformation [15] splits tensors and uses asynchronous swapping to hide the data transfer latency. Unlike previous work, EDA adjusts operator traversal depth in a subgraph of a DNN model to minimize the off-chip memory traffic of an NPU.

Memory Systems of Accelerators: The memory system of the NPU [47], [50] often consumes a large portion of energy. The prior work [24], [25], [27], [31], [43], [52], [54], [55] smartly swaps the large weight data between the off-chip and on-chip memory to reduce GPU energy consumption. In addition, the prior tensor-splitting work [17], [28], [45], [49], [53], [66] intelligently breaks tensors into multiple small pieces to reduce the latency and peak usage of memory systems in modern NPUs. The prior work [8], [12], [23], [33], [36], [38], [44], [56] also exhibits significant data locality in DNN models and updates the accelerator architecture and system. EDA transforms a DNN model into a tensor-splitting model. The small tensor reduces the peak usage of on-chip memory [65]. Finally, EDA reuses the outputs of operators and reduces the off-chip memory traffic on an NPU.

TABLE IV
EDA COMPILATION TIME AT EACH STAGE (SECONDS)

	GRAPHING	SUBGRAPH	GROUP	SCHED	TOTAL
RES [11]	1.74	9.68	0.004	0.02	11.44
MNT [11]	0.38	2.21	0.003	0.02	2.61
EDSR [4]	4.04	4.11	0.01	1.57	9.73
UNET [3]	10.92	3.52	0.02	1.13	15.59
ENT [11]	0.49	10.87	0.01	0.03	11.4
SESR [11]	1.21	0.08	0.01	0.39	1.69
YOLO [5]	0.94	0.8	0.01	0.03	1.78
BERT [7]	0.05	1.63	0.01	0.01	1.7

VIII. CONCLUSION

This paper creates a novel DNN model compiler called *EDA* that reschedules operators to improve the energy efficiency of the NPU on edge devices. The traditional layer-wise operator scheduler on the NPU reuses intermediate tensors of operators on the on-chip memory space with great difficulty, substantially increasing the off-chip memory data traffic. The inter-layer operator scheduler increases the intermediate tensor data reuse rate on the on-chip memory space, but its deep operator traversal path also significantly increases off-chip memory traffic as the size of memory space on the NPU decreases. To resolve the issues around the previous scheduling methods, our approach decomposes a model graph into multiple subgraphs and considers the available on-chip memory size to adjust the depth of the inter-layer operator traversal path. Furthermore, EDA transparently performs model transformation, operator scheduling, and tensor memory allocation. Our evaluation shows our approach that achieves a geometric mean of $2.39\times$ speedup, improving the $2.08\times$ energy efficiency over the NPU with designated on-chip memory buffers.

IX. ACKNOWLEDGEMENT

This work is supported by the grants from Google and the National Science and Technology Council in Taiwan (111-2221-E-A49-131-MY3) (113-2640-E-A49-004-).

REFERENCES

- [1] "Planaria: Spatially Multi-Tenant DNN Acceleration through Dynamic Architecture Fission Simulator," [Online]. Available: <https://github.com/he-actlab/planaria.code>, 2020, (accessed April. 16, 2024).
- [2] "AMD Ryzen™ Threadripper™ PRO 5965WX Processor Spec." [Online]. Available: <https://www.amd.com/en/products/cpu/amd-ryzen-threadripper-pro-5965wx>, 2023, (accessed April. 16, 2024).
- [3] "Satellite Image Segmentation with UNet and Its Variants," [Online]. Available: <https://github.com/ashishpatel26/satellite-Image-Semantic-Segmentation-Unet-Tensorflow-keras>, 2023, (accessed April. 16, 2024).
- [4] "Single Image Super-Resolution with EDSR, WDSR and SRGAN," [Online]. Available: <https://github.com/krasserm/super-resolution>, 2023, (accessed April. 16, 2024).
- [5] "STM32 model zoo," [Online]. Available: <https://github.com/STMicroelectronics/stm32ai-modelzoo>, 2023, (accessed April. 16, 2024).
- [6] D. Abts, J. Ross, J. Sparling, M. Wong-VanHaren, M. Baker, T. Hawkins, A. Bell, J. Thompson, T. Kahsai, G. Kimmell *et al.*, "Think fast: A tensor streaming processor (tsp) for accelerating deep learning workloads," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 145–158.
- [7] AI-Benchmark, "AI Benchmark V6 Mobile," [Online Available:] <https://ai-benchmark.com/download>, 2023, (accessed Oct. 16, 2024).
- [8] B. Akin, Z. A. Chishti, and A. R. Alameldeen, "Zcomp: Reducing dnn cross-layer memory footprint using vector extensions," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2019, pp. 126–138.
- [9] M. Alwani, H. Chen, M. Ferdman, and P. Milder, "Fused-layer cnn accelerators," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2016, pp. 1–12.
- [10] ARM Ltd., "Miniaturizing Models for microNPUs: a Cascading Scheduler," [Online Available:] <https://www.youtube.com/watch?v=4MV0wt5YLo>, 2022, (accessed Jan. 20, 2024).
- [11] ARM Ltd., "Arm ML Model Zoo," [Online]. Available: <https://github.com/ARM-software/ML-zoo>, 2023, (accessed April. 16, 2024).
- [12] A. Azizimazreah and L. Chen, "Shortcut mining: Exploiting cross-layer shortcut reuse in dcnn accelerators," in *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2019, pp. 94–105.
- [13] P. G. Bradford, G. J. Rawlins, and G. E. Shannon, "Efficient matrix chain ordering in polylog time," *SIAM Journal on Computing*, vol. 27, no. 2, pp. 466–490, 1998.
- [14] J. Cai, Y. Wei, Z. Wu, S. Peng, and K. Ma, "Inter-layer scheduling space definition and exploration for tiled accelerators," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2023, pp. 1–17.
- [15] R. Chen, Z. Ding, S. Zheng, C. Zhang, J. Leng, X. Liu, and Y. Liang, "Magis: Memory optimization via coordinated graph transformation and scheduling for dnn," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operation Systems (ASPLOS)*, 2024, pp. 607–621.
- [16] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze *et al.*, "{TVM}: An automated {End-to-End} optimizing compiler for deep learning," 2018, pp. 578–594.
- [17] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 16 344–16 359, 2022.
- [18] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, T. Wang *et al.*, "Tensorflow lite micro: Embedded machine learning for tinyml systems," *Proceedings of Machine Learning and Systems*, vol. 3, pp. 800–811, 2021.
- [19] FreePDK45, "FreePDK45," <https://eda.ncsu.edu/freepdk/freepdk45/>, 2011.
- [20] M. Gao, X. Yang, J. Pu, M. Horowitz, and C. Kozyrakis, "Tangram: Optimized coarse-grained dataflow for scalable nn accelerators," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operation Systems (ASPLOS)*, 2019, pp. 807–820.
- [21] S. Ghodrati, B. H. Ahn, J. K. Kim, S. Kinzer, B. R. Yatham, N. Alla, H. Sharma, M. Alian, E. Ebrahimi, N. S. Kim *et al.*, "Planaria: Dynamic architecture fission for spatial multi-tenant acceleration of deep neural networks," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2020, pp. 681–697.
- [22] K. Han, A. Xiao, E. Wu, J. Guo, C. Xu, and Y. Wang, "Transformer in transformer," *Advances in neural information processing systems (NeurIPS)*, vol. 34, pp. 15 908–15 919, 2021.
- [23] K. Hegde, J. Yu, R. Agrawal, M. Yan, M. Pellauer, and C. Fletcher, "Ucnn: Exploiting computational reuse in deep neural networks via weight repetition," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2018, pp. 674–687.
- [24] M. Hildebrand, J. Khan, S. Trika, J. Lowe-Power, and V. Akella, "Autotm: Automatic tensor movement in heterogeneous memory systems using integer linear programming," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operation Systems (ASPLOS)*, 2020, pp. 875–890.
- [25] C.-C. Huang, G. Jin, and J. Li, "Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operation Systems (ASPLOS)*, 2020, pp. 1341–1355.
- [26] Q. Huang, M. Kang, G. Dinh, T. Norell, A. Kalaiah, J. Demmel, J. Wawrzyniek, and Y. S. Shao, "Cosa: Scheduling by constrained optimization for spatial accelerators," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2021, pp. 554–566.
- [27] A. Ivanov, N. Dryden, T. Ben-Nun, S. Li, and T. Hoefler, "Data movement is all you need: A case study on optimizing transformers," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 3, pp. 711–732, 2021.
- [28] T. Jin and S. Hong, "Split-cnn: Splitting window-based operations in convolutional neural networks for memory system optimization," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operation Systems (ASPLOS)*, 2019, pp. 835–847.
- [29] N. P. Jouppi, D. H. Yoon, M. Ashcraft, M. Gottscho, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. Ma, X. Ma *et al.*, "Ten lessons from three generations shaped google's tpuv4i: Industrial product," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2021, pp. 1–14.
- [30] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1–12.
- [31] J. Jung, J. Kim, and J. Lee, "Deepum: Tensor migration and prefetching in unified memory," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operation Systems (ASPLOS)*, 2023, pp. 207–221.
- [32] S.-C. Kao, S. Subramanian, G. Agrawal, A. Yazdanbakhsh, and T. Krishna, "Flat: An optimized dataflow for mitigating attention bottlenecks," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operation Systems (ASPLOS)*, 2023, pp. 295–310.
- [33] H. Kim, S. Ahn, Y. Oh, B. Kim, W. W. Ro, and W. J. Song, "Duplo: Lifting redundant memory accesses of deep neural networks for gpu tensor cores," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 725–737.
- [34] J. Kim, S. Na, S. Lee, S. Lee, and J. Huh, "Improving data reuse in npu on-chip memory with interleaved gradient order for dnn training," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2023, pp. 438–451.
- [35] M. Kim, "Evolutionary migration from lpdrr3 to lpdrr4," in *JEDEC, Tech. Report*, 2014.
- [36] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of dnn dataflow: A data-centric approach," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2019, pp. 754–768.
- [37] H. Kwon, A. Samajdar, and T. Krishna, "Maeri: Enabling flexible dataflow mapping over dnn accelerators via reconfigurable interconnects," *ACM SIGPLAN Notices*, vol. 53, no. 2, pp. 461–475, 2018.
- [38] Y. Kwon, Y. Lee, and M. Rhu, "Tensordimm: A practical near-memory processing architecture for embeddings and tensor operations in deep learning," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2019, pp. 740–753.
- [39] C.-J. Lee and T. T. Yeh, "Resa: Reconfigurable systolic array for multiple tiny dnn tensors," *ACM Transactions on Architecture and Code Optimization*, vol. 21, no. 3, pp. 1–24, 2024.

- [40] S. Lee, H. Cho, Y. H. Son, Y. Ro, N. S. Kim, and J. H. Ahn, "Leveraging power-performance relationship of energy-efficient modern dram devices," *IEEE Access*, vol. 6, pp. 31 387–31 398, 2018.
- [41] S. Lee, J. Kim, S. Na, J. Park, and J. Huh, "Tnpu: Supporting trusted execution with tree-less integrity protection for neural processing unit," in *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 229–243.
- [42] J. Lin, W.-M. Chen, Y. Lin, C. Gan, S. Han *et al.*, "Mcunet: Tiny deep learning on iot devices," *Advances in neural information processing systems (NeuralPS)*, vol. 33, pp. 11 711–11 722, 2020.
- [43] S.-F. Lin, Y.-J. Chen, H.-Y. Cheng, and C.-L. Yang, "Tensor movement orchestration in multi-gpu training systems," in *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 1140–1152.
- [44] J. Liu, F. Zhang, J. Guan, H.-H. Sung, X. Guo, X. Du, and X. Shen, "Space-efficient trec for enabling deep learning on microcontrollers," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operation Systems (ASPLOS)*, 2023, pp. 644–659.
- [45] Y.-Y. Liu, H.-S. Zheng, Y. F. Hu, C.-F. Hsu, and T. T. Yeh, "Tinys: Memory-efficient tinyml model compiler framework on microcontrollers," in *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 848–860.
- [46] W. Lu, G. Yan, J. Li, S. Gong, Y. Han, and X. Li, "Flexflow: A flexible dataflow accelerator architecture for convolutional neural networks," in *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*, IEEE, 2017, pp. 553–564.
- [47] K. T. Malladi, B. C. Lee, F. A. Nothaft, C. Kozyrakis, K. Periyathambi, and M. Horowitz, "Towards energy-proportional datacenter memory with mobile dram," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2012, p. 37–48.
- [48] N. Muralimanohar, R. Balasubramonian, and N. P. Jouppi, "Cacti 7.0: A tool to model large caches," *HP laboratories*, vol. 27, p. 28, 2009.
- [49] X. Nie, X. Miao, Z. Yang, and B. Cui, "Tsplint: Fine-grained gpu memory management for efficient dnn training via tensor splitting," in *2022 IEEE 38th International Conference on Data Engineering (ICDE)*, 2022, pp. 2615–2628.
- [50] M. O'Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, "Fine-grained dram: Energy-efficient dram for extreme bandwidth systems," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2017, pp. 41–54.
- [51] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A systematic approach to dnn accelerator evaluation," in *2019 IEEE international symposium on performance analysis of systems and software (ISPASS)*, 2019, pp. 304–315.
- [52] X. Peng, X. Shi, H. Dai, H. Jin, W. Ma, Q. Xiong, F. Yang, and X. Qian, "Capuchin: Tensor-based gpu memory management for deep learning," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operation Systems (ASPLOS)*, 2020, pp. 891–905.
- [53] I. Prakash, A. Bansal, R. Verma, and R. Shorey, "Smartsplit: Latency-energy-memory optimisation for cnn splitting on smartphone environment," in *2022 14th International Conference on Communication Systems & NETWORKS (COMSNETS)*, 2022, pp. 549–557.
- [54] J. Ren, J. Luo, K. Wu, M. Zhang, H. Jeon, and D. Li, "Sentinel: Efficient tensor migration and allocation on heterogeneous memory systems for deep learning," in *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 598–611.
- [55] M. Rhu, N. Gimelshein, J. Clemons, A. Zulfiqar, and S. W. Keckler, "vdnn: Virtualized deep neural networks for scalable, memory-efficient neural network design," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2016, pp. 1–13.
- [56] M. Riera, J.-M. Arnau, and A. González, "Computation reuse in dnns by exploiting input similarity," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2018, pp. 57–68.
- [57] A. Sabne, "Xla: Compiling machine learning for peak performance," *Google Res*, 2020.
- [58] Y. S. Shao, J. Clemons, R. Venkatesan, B. Zimmer, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina *et al.*, "Simba: Scaling deep-learning inference with multi-chip-module-based architecture," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2019, pp. 14–27.
- [59] Y. N. Wu, J. S. Emer, and V. Sze, "Accelergy: An architecture-level energy estimation methodology for accelerator designs," in *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2019, pp. 1–8.
- [60] Y. N. Wu, P.-A. Tsai, A. Parashar, V. Sze, and J. S. Emer, "Sparseloop: An analytical approach to sparse tensor accelerator modeling," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2022, pp. 1377–1395.
- [61] Q. Xiao, S. Zheng, B. Wu, P. Xu, X. Qian, and Y. Liang, "Hasco: Towards agile hardware and software co-design for tensor computation," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2021, pp. 1055–1068.
- [62] Y. Yang, J. S. Emer, and D. Sanchez, "Isosceles: Accelerating sparse cnns through inter-layer pipelining," in *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 598–610.
- [63] A. Yazdanbakhsh, K. Seshadri, B. Akin, J. Laudon, and R. Narayanaswami, "An evaluation of edge tpu accelerators for convolutional neural networks," *arXiv preprint arXiv:2102.10423*, vol. 1, no. 6, 2021.
- [64] T. T. Yeh, M. D. Sinclair, B. M. Beckmann, and T. G. Rogers, "Deadline-aware offloading for high-throughput accelerators," in *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 479–492.
- [65] B. Zheng, N. Vijaykumar, and G. Pekhimenko, "Echo: Compiler-based gpu memory footprint reduction for lstm rnn training," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2020, pp. 1089–1102.
- [66] H.-S. Zheng, Y.-Y. Liu, C.-F. Hsu, and T. T. Yeh, "Streamnet: Memory-efficient streaming tiny deep learning inference on the microcontroller," *Advances in Neural Information Processing Systems (NeuralPS)*, vol. 36, pp. 37 160–37 172, 2024.
- [67] S. Zheng, X. Zhang, L. Liu, S. Wei, and S. Yin, "Atomic dataflow based graph-level workload orchestration for scalable dnn accelerators," in *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 475–489.
- [68] S. Zheng, R. Chen, A. Wei, Y. Jin, Q. Han, L. Lu, B. Wu, X. Li, S. Yan, and Y. Liang, "Amos: enabling automatic mapping for tensor computations on spatial accelerators with hardware abstraction," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2022, pp. 874–887.